

K-Nearest Neighbors(KNN) Algorithm

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks.

It works by finding the "k" closest data points (neighbors) to a given input and makes a prediction based on the majority class (for classification) or the average value (for regression).

KNN is a non-parametric, lazy learning algorithm

Non-parametric

- KNN does not assume a model form like linear regression does (no line or curve).
- It simply stores training data and uses distance to classify or predict a value.

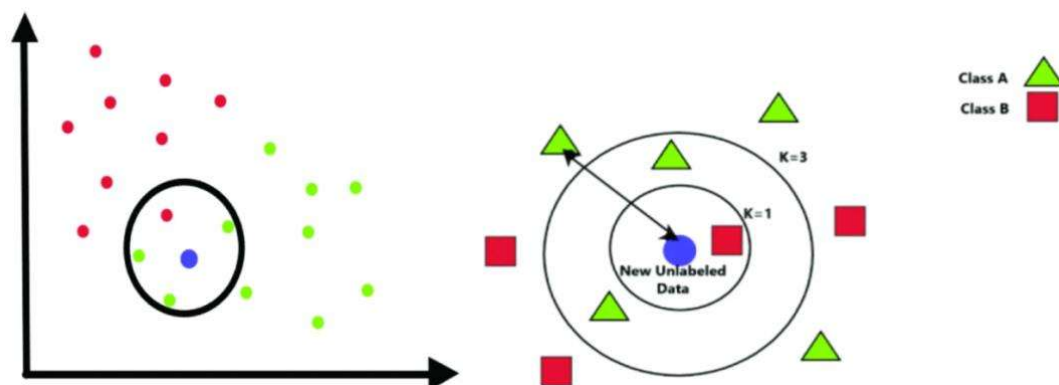
In machine learning language, KNN is *non-parametric* because it does not summarize the data with fixed parameters.

Lazy learner

- KNN doesn't build a model in training — it *waits until prediction time*.
- All work happens during prediction.

This is called “lazy learning”.

K-Nearest Neighbors is also called as a lazy learner algorithm because it does not learn from the training set immediately instead it stores the entire dataset and performs computations only at the time of classification.



WHY DISTANCE MATTERS

KNN decisions depend entirely on distance.

KNN algorithm

1. Picks a new data point
2. Measures distance to all training points
3. Selects **k closest neighbors**
4. Majority vote (classification), average value (regression)

The most common way to measure this distance is by using the **Euclidean distance (default in KNN) and Manhattan Distance**. Once the distances are calculated, the algorithm takes a majority vote from the K-nearest points and classifies the new point accordingly.

In KNN, distance means: How similar are two data points?

- Small distance → very similar
- Large distance → very different

Everything KNN does depends on this.

MANHATTAN DISTANCE

$$\text{Manhattan Distance} = \sum_{i=1}^n |x_1^i - x_2^i|$$

- n – number of features in the dataset
- x1 and x2 are the two data points between which distance is calculated
- Look at each feature one by one
- Take the **absolute difference**
- Add everything together

Example dataset:

Height (cm)	Weight (kg)
170	65
175	70

- Feature 1 = Height
- Feature 2 = Weight

So each data point is a **vector**:

Person A = (170, 65)
Person B = (175, 70)

Compare them **feature by feature (column by column)**.

Compare first feature (Height)

$$|170 - 175| = 5$$

Compare second feature (Weight)

$$|65 - 70| = 5$$

Add them

$$5 + 5 = 10$$

That's **Manhattan distance**.

Manhattan distance is like walking in a city grid.

You can only:

- Go left/right
- Go up/down
- you could only move along horizontal and vertical lines like a grid or city streets. It's also called "taxicab distance" because a taxi can only drive along the grid-like streets of a city.

You cannot cut diagonally.

That's why it's called *Manhattan* (New York street grid).

EUCLIDEAN DISTANCE

Formula

$$\text{Euclidean Distance} = \sqrt{\sum_{i=1}^n (x_1^i - x_2^i)^2}$$

Same idea — still feature by feature — just combined differently.

- Look at the difference for each feature
- Square it (makes everything positive)
- Add them
- Take square root

Step 1: Height difference

$$(170 - 175)^2 = 25$$

Step 2: Weight difference

$$(65 - 70)^2 = 25$$

Step 3: Add and square root

$$\sqrt{(25 + 25)} = \sqrt{50} \approx 7.07$$

Euclidean distance is **straight-line distance** between two points.

-the shortest path you would walk if you were to go directly from one point to another.

Cutting diagonally across space

```
# Euclidean (default)
KNeighborsClassifier(n_neighbors=5, metric='euclidean')

# Manhattan
KNeighborsClassifier(n_neighbors=5, metric='manhattan')
```

Euclidean works well when Dimensionality is small to medium while Manhattan often works better than Euclidean as dimensionality becomes high.

Feature Scaling

One important step in using KNN is feature scaling.

If one feature is much larger in value than another, it can dominate the distance calculation. For example, if we're using both age and income as features to classify a person's group, the income in thousands could make the results biased. To avoid this, we normalize the data so that each feature has equal weight.

Example: Different features have: Different units, Different ranges

Imagine:

- Feature 1: **Age** (0–100)
- Feature 2: **Salary** (0–100,000)

A = (35, 30000)

$B = (40, 40000)$

$$\begin{aligned}(35 - 40)^2 &= (5)^2 = 25 \\ (30000 - 40000)^2 &= (10000)^2 = 100000000 \\ \sqrt{25 + 100000000} &= \sqrt{100000025}\end{aligned}$$

Which feature dominates distance?

Distance $\approx$ salary difference
Age becomes almost irrelevant

This is unfair distance.

Solution

Feature Scaling

```
# Example Implementation
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

StandardScaler works column by column, using the whole dataset
StandardScaler first computes the mean and standard deviation of *each feature (column)*, separately.

Assume the dataset looks like this:

Age column

Ages: 25, 30, 35, 38, 40, 45

- **Mean Age ≈ 35**
- **Standard deviation Age ≈ 5**

Interpretation: Many ages lie within 35 ± 5

“A typical age is about 5 years away from the mean age..”

Standard deviation measures On average, how far are data points from the mean?

Salary column

Salaries: 20000, 25000, 30000, 31000, 40000

- **Mean Salary $\approx 30,000$**
- **Std Salary $\approx 10,000$**

A typical salary difference is about 10,000

Then it Scales each value so that the transformed feature has mean 0 and standard deviation 1. i.e. the average distance of values from the mean (0) is 1.

$$\text{scaled value} = \frac{\text{value} - \text{mean}}{\text{std}}$$

Scale AGE

Person A (Age 35)

$$(35 - 35) / 5 = 0$$

Person B (Age 40)

$$(40 - 35) / 5 = 1$$

- A is exactly average age
- B is **1 age-step** above average

Scale SALARY

Person A (Salary 30000)

$$(30000 - 30000) / 10000 = 0$$

Person B (Salary 40000)

$$(40000 - 30000) / 10000 = 1$$

Meaning:

- A has average salary
- B is **1 salary-step** above average

New scaled points

After StandardScaler:

A → (0 , 0)
B → (1 , 1)

Distance after scaling (now it's fair)

Feature-wise differences (scaled)

- Age difference = $|0 - 1| = 1$
- Salary difference = $|0 - 1| = 1$

Euclidean distance

$$\sqrt{(1^2 + 1^2)}$$

Age and salary now contribute equally to distance, because they are equally unusual relative to their normal variation.

StandardScaler makes different features comparable by measuring how unusual a value is, not how large it is.

- Mean → defines the center
- Std → defines a typical step size
- Scaling → removes unit dominance
- Distance → becomes meaningful

After StandardScaler, features do NOT have a fixed range.
But most values usually fall between -2 and +2.

Original features

- Age: 0 – 100
- Salary: 0 – 100,000

After scaling:

- Units disappear
- Ranges are no longer “years” or “euros”

Typical new ranges

Because of how real data behaves:

Scaled value	Meaning
0	Average
±1	One standard deviation away
±2	Quite unusual
±3	Very rare

So in practice:

Most scaled Age and Salary values will lie roughly between -2 and $+2$

We had:

- Mean Age = 35, std = 5
- Mean Salary = 30,000, std = 10,000

Age examples after scaling

Age	Scaled value
35	0
40	+1
30	-1
45	+2
25	-2

So new Age range:

≈ -2 to $+2$

Salary examples after scaling

Salary	Scaled value
30,000	0
40,000	+1
20,000	-1
50,000	+2
10,000	-2

So new Salary range:

≈ -2 to $+2$

What happens with outliers?

If someone has:

- Age = 60
- Salary = 90,000

Scaled values might be:

Age $\rightarrow +5$

Salary $\rightarrow +6$

That is allowed.

StandardScaler does not clip values.

Why this makes distances fair

Distance algorithms assume:

All features are equally important.” But raw data violates this assumption.

StandardScaler fixes that by:

- Removing unit differences (cm, €, kg)
- Making all features comparable

How to choose the value of k for KNN Algorithm?

- The value of k in KNN decides how many neighbors the algorithm looks at when making a prediction.
- Choosing the right k is important for good results.
- Using a larger k reduces the effect of noise and outliers by averaging over more neighbours.
- But if k is too large, the model becomes overly simple, ignores local patterns, and underfits the data.
- So k should be picked carefully based on the data

Cross validation

What problem does cross-validation solve?

If we train and test only once:

- The result may depend on luck (which data went into test set)
- Accuracy may be too optimistic or too pessimistic

What's important is how well does this model perform in general, not just on one split?

That's exactly what cross-validation answers.

Cross-validation tests the model multiple times on different parts of the data and averages the results.

k-Fold Cross-Validation

Assume:

- You have 100 data points
- You choose $k = 5$ folds

Split the data

Divide the dataset into 5 equal parts (folds):

Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5

Train & test multiple times

You repeat this process **5 times**:

Run	Training data	Test data
1	Folds 2–5	Fold 1
2	Folds 1,3,4,5	Fold 2
3	Folds 1,2,4,5	Fold 3
4	Folds 1–3,5	Fold 4
5	Folds 1–4	Fold 5

Each fold is used once as test data.

Average the results

You get 5 accuracy scores:

0.82, 0.80, 0.83, 0.81, 0.82

Final cross-validation accuracy:

Average $\approx$ 0.82

This is more reliable than a single train–test split.

Because every data point gets a chance to be in the test set.

So:

- Less bias
- Less dependence on one lucky split
- Better estimate of real performance

Common values of k

k (folds)	When to use
5	Default, good balance
10	More reliable, slower
n (Leave-One-Out)	Very small datasets